



Continuous Select-and-Prune Incremental Learning for Encrypted Traffic Classification in Distributed SDN Networks

Son Duong, Hai-Anh Tran, Truong X Tran

► To cite this version:

Son Duong, Hai-Anh Tran, Truong X Tran. Continuous Select-and-Prune Incremental Learning for Encrypted Traffic Classification in Distributed SDN Networks. 2024 IEEE 49th Conference on Local Computer Networks (LCN), Oct 2024, Normandy, France. pp.1 - 8, 10.1109/lcn60385.2024.10639717 . hal-04722755

HAL Id: hal-04722755

<https://hal.science/hal-04722755v1>

Submitted on 6 Oct 2024

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Continuous Select-and-Prune Incremental Learning for Encrypted Traffic Classification in Distributed SDN Networks

Son Duong, Hai-Anh Tran[✉]

School of Information and Communications Technology
Hanoi University of Science and Technology (HUST)
Hanoi, Vietnam

son.dc232248M@sis.hust.edu.vn, anhth@soict.hust.edu.vn

Truong X. Tran[✉]

School of Science, Engineering and Technology
Penn State Harrisburg, The Pennsylvania State University
Middletown, PA 17057, United States
truong.tran@psu.edu

Abstract—Traffic classification plays an indispensable role in Computer Networks and the Internet of Things. As the cybersecurity landscape evolves, a diverse array of encrypted protocols (e.g., HTTPS, GQUIC, and TLS) is becoming increasingly prevalent. Alongside this, the challenge of encrypted traffic classification has garnered renewed attention, fostered by the increasing adoption of Deep Learning (DL) methodologies. Nonetheless, the fast-paced release of new encrypted protocols necessitates frequent retraining of DL models on reformed datasets encompassing encrypted traffic from both known and unknown applications. This requirement can lead to the issues of catastrophic forgetting, particularly when classifying unknown applications. To address this shortcoming, we propose a novel two-stage Incremental Learning (IL) paradigm based on flow-exemplar selection strategy and model pruning, CoSP, to enable continuous model evolution with unknown applications. Extensive experiments on encrypted traffic datasets in a Software-defined networking environment illustrate that our method outperforms other IL approaches, achieving 1.07% and 0.94% improvements in last accuracy and forgetting, respectively.

Index Terms—Encrypted traffic classification, SDN, Incremental Learning, Machine Learning.

I. INTRODUCTION

Network traffic classification is crucial for network management, resource allocation, user behavior analysis, and security monitoring [1]. According to the Annual Report of Let's Encrypt 2023 [2], HTTPS page loads globally have increased, indicating the prevalent trend of encrypted traffic. Consequently, methods like Port-based and data packet inspection (DPI), once effective with unencrypted traffic, are now rendered ineffective [3].

With the proliferation of deep learning methods, several machine learning (ML)/deep learning (DL) solutions for encrypted traffic classification (ETC) [4], [5] have been deployed in intelligent computer networks. However, integrating DL mechanisms into traditional networks is not straightforward. Software-defined networking (SDN) [6] emerges with the aim of eliminating the static and distributed control architecture of traditional networks, instead creating a centralized control architecture with a flexible, easily manageable, and fault-tolerant network.

In ML-and DL-based approaches for ETC, nonetheless, a common hurdle arises from their reliance on static models trained on predetermined classes and fixed feature sets within closed-world environments. In the context of distributed SDN, these models encounter a practical challenge: *When a new SDN domain emerges within a dynamic-network environment, they encounter difficulties in adapting classifiers to unknown classes*, as sketched in Fig. 1. The predominant methods typically merge known and unknown classes to update the model. Yet, retraining the model entirely with extensive data proves redundant, time-intensive, and economically burdensome [7].

In response to the challenge of static models, the research community is increasingly interested in applying Incremental Learning (IL) techniques to ETC [8], [9], [10], [11], [12]. Specifically, IL, also known as continuous or online learning, is an effective strategy for learning unknown classes and retaining previous knowledge. This learning paradigm presents a promising solution to the challenges arising from the dynamic network environment, as it enables efficient model updates and maintains accurate classification performance. The aforementioned studies commonly tackle ETC problems akin to those in computer vision applications. However, the informational richness of encrypted flows surpasses that of images. Simultaneously, within the context of real-world ETC scenarios, data within the new network domain frequently encompass both known and unknown patterns. This demands the establishment of policies to effectively exploit data and insights derived from previous tasks.

The shortcomings of those studies were the motivation for this paper to introduce an IL framework based on **Continuous Select-and-Prune** methodology (**CoSP**) to keep new ETC model (in target domain) evolving with (un)known encrypted traffic. The main novelty of CoSP encompasses three key aspects: 1) Proposing a flow-exemplar selection algorithm to construct/update an effective exemplar set. This algorithm considers the trade-off between exploitation (selecting high-quality samples via herding selection) and exploration (generating samples using a Generative Adversarial Network (GAN)); 2) Implementing a model pruning technique that

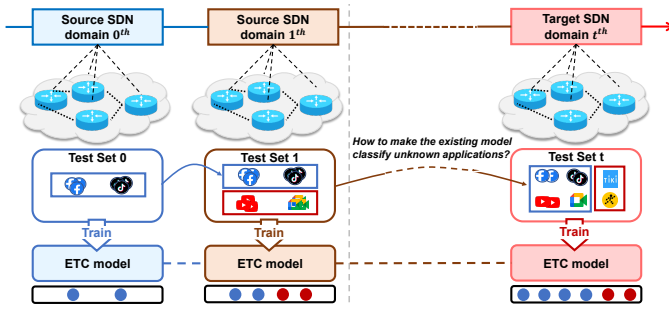


Fig. 1: Illustration depicting the continuous emergence of SDN domain scenarios. The ETC model necessitates adept adaptation to unknown applications (red dots) while retaining knowledge of previous applications (blue dots) without experiencing catastrophic forgetting.

leverages pruned knowledge from previous ETC models in source domains to control over-parameterized expansion during training; 3) Computing a General Compositional Loss aimed at balancing accuracy and forgetting metrics.

Our contribution can be briefly summarized as follows:

- **Proposing CoSP for (un)known ETC:** An IL framework, CoSP, maintains the classification of (un)known applications by ETC models without requiring training from scratch. Proposed techniques, such as flow-exemplar selection and model pruning, are deployed to adapt to real-world ETC scenarios, avoiding redundant storage.
- **Incorporating CoSP into ETC for distributed SDN networks:** The CoSP framework is implemented and deployed within the controller plane of the SDN architecture. The efficacy of CoSP is assessed using both the GQUIC dataset and a dataset collected internally.

The remainder of this paper is structured as follows. Section II provides a survey of existing DL and IL methods for ETC. Section III presents the problem definition and an overview of the CoSP framework. The experiments and evaluations are detailed in Section IV. Finally, Section V provides the conclusion and potential future work.

II. RELATED WORKS

This section conducts a comprehensive analysis of the ETC problem, employing two distinct approaches: DL without IL and DL with IL.

A. Deep Learning approaches without Incremental Learning

The advancements in DL-based approaches have demonstrated excellent performance on ETC problems, leveraging deep learning techniques, such as recurrent neural networks, convolutional neural networks (CNN), and attention mechanisms. Furthermore, DL enables the automatic extraction of complex features from encrypted traffic for efficient classification without needing expert network knowledge.

Lin *et al.* [4] introduced ET-BERT, a traffic representation model based on bidirectional encoder representations transformer, achieving state-of-the-art results with improved f1-scores on various real-world network datasets. He *et al.* [5]

developed PERT, utilizing dynamic word embedding to handle encoded values of network packets. PERT outperformed benchmarks like 1D-CNN and 2D-CNN on datasets such as Android HTTPS Traffic and ISCX2016 VPN-non VPN.

Shapira *et al.* [13] designed an end-to-end CNN model, FlowPIC, for ETC, encoding packet information into flow representations and feeding them into a CNN. Lim *et al.* [14] implemented a network traffic classification model within SDN using DL techniques like multi-layer LSTMs and hybrid CNN-LSTM models, achieving classification with payload sizes ranging from 36 to 512 bytes. Lotfollahi *et al.* [3] (2020) proposed Deep Packet, a framework involving feature extraction and classification using Stacked Autoencoder (SAE) and 1DCNN. Their model achieved high accuracy rates on an open dataset of 200,000 packets across 15 applications, with recall scores of 0.98 in SAE and 0.94 in 1DCNN.

Most existing ETC methods rely on automatic feature selection techniques rather than manual methods, which can be time-consuming and prone to errors [3]. However, these approaches are often limited to categorizing applications within predefined sets. This limitation becomes impractical as new applications and traffic streams emerge rapidly, making unknown applications unclassifiable by traditional DL methods. Therefore, there is a pressing need for DL models that can automatically learn internal features using ETE blocks (Section III-C1), and additionally employ an IL framework to classify both unknown and known applications.

B. Approaches using Incremental Learning

The objective of incremental learning is to handle a continuous stream of data, mitigating catastrophic forgetting while ensuring the stability and plasticity of the ETC model.

Li *et al.* [8] introduced MISS, an IL framework employing multi-view sequence fusion to adapt ETC models to unknown applications in dynamic network environments. MISS showcased three key features: enhanced knowledge extraction, an exemplar selection algorithm to reduce redundancy, and the integration of new hyperparameters and loss functions. Results exhibited MISS's superiority over existing IL methods for ETC, achieving significant improvements of 11.37% and 1.58% on real-world network traffic datasets. Bovenzi *et al.* [9] enhanced iCaRL (Incremental classifier and representation learning) by implementing a technique to retain the knowledge of prior classes with a limited number of stored samples. Their assessment utilized the MIRAGE19 dataset, encompassing traffic data from 40 widely-used Android applications, thereby ensuring replicability. As a result, the classification performance notably declined with varying numbers of IL episodes and the addition of unknown applications per episode. Wu *et al.* [10] proposed a framework that utilized knowledge distillation and bias correction to tackle the challenge of catastrophic forgetting in standard CNN models. The results showed faster and more precise traffic classification, accompanied by a notable reduction in time and memory consumption by approximately 50% compared to benchmark algorithms. Chen *et al.* [11] utilized an IL framework, incorporating

the one vs rest (OvR) strategy and neural network classifiers, to extend the binary Support Vector Machine (SVM) model for classifying unknown applications. Addressing the issue of scalability with a large number of traffic classes, their approach minimized memory and computational resource consumption. Experimental results demonstrated that their framework attained high classification accuracy comparable to closed-world methods. Furthermore, the selection algorithm notably streamlined training efforts, ensuring dataset scale control and meeting classification accuracy criteria in lifelong IL scenarios. Lee *et al.* [12] utilized an incremental SVM with Stochastic Gradient Descent to detect encrypted malware traffic. Their method incorporated periodic updates to sustain performance levels, utilizing incremental algorithms trained on 31 flow features extracted from Transport Layer Security (TLS), Hypertext Transfer Protocol (HTTP), and Domain Name System. The findings underscored the superiority of their incremental algorithm for detection, showcasing high accuracy both offline and online with a low false discovery rate.

Existing studies often approach encrypted traffic classification (ETC) using techniques similar to those applied in computer vision. However, encrypted network flows contain richer information than images, and real-world scenarios typically involve a limited number of unknown classes compared to known ones. Therefore, in addition to proposed imitation learning (IL) techniques such as pruning and distillation, this paper also presents a strategy for selecting a comprehensive and suitable exemplar set to address the ETC problem (refer to Section III-B).

III. METHODOLOGY

A. Problem Definition and CoSP solutions

1) Problem Definition

Consider a distributed SDN environment with a set S^A of N_s available source domains ($|S^A| = N_s$). We denote S^t as the target domain that is newly established in the network. For each domain S^i , we can establish a network traffic dataset $D^i = \{X_j^i, y_j^i\}_{j=1}^{N_i}$ is composed of N^i pairs of flow-feature matrix X_j^i (described in Section IV-A2) and application $y_j^i \in \mathcal{Y}^i$. Also, $Y = \{y_j^i\}_{j=1}^{N_y}$ denotes the set of applications (e.g., Facebook, TikTok, Youtube) in i^{th} incremental class.

The problem of training the ETC model Θ^t is as follows:

Input: The source datasets $D^A = \{D^i\}_{i=1}^{N_s}$, ETC models trained for previous tasks (domains) $\Theta^A = \{\Theta^i\}_{i=1}^{N_s}$, and the target dataset D^t .

Output: An ETC model Θ^t , which is trained to classify the unknown classes in the target domain without forgetting or losing the knowledge learned from the source domains.

2) CoSP solutions

CoSP addresses ETC on application planes in a distributed SDN system. Each SDN domain preprocesses application features in its control plane before deploying them to the data plane for CoSP execution. Fig. 2 illustrates the overall framework of the proposed CoSP model learned at the $(t-1)^{th}$ and t^{th} tasks (domains), designated as the source

and target models (i.e., Θ^{t-1} and Θ^t). CoSP will effectively select exemplar flow samples from the source domains (*Flow-exemplar Selection block*), automatically extract important features (*Feature Extractor block*), leverage pruned knowledge from Θ^{t-1} (*ETC model pruning block*), and train Θ^t with minimal the target loss (*General compositional loss*).

In the target SDN domain S^t , the dataset $D^t = \{E^t \cup T^t\}$ is a combination of exemplar flows set E^t from the source domains S^A and newly target flows set T^t in the target domain S^t .

In terms of the exemplar set E^t , the number of flows in the set E^t , denoted as N_E^t , is determined by $N_E^t = \beta \cdot N_T^t$. The *old-new imbalance ratio* β serves as the balancing hyperparameter between the number of flows in the D^t set and those in the E^t set. The construction and update of E^t leverage flows from the source domain S^{t-1} through the *Flow-exemplar Selection* stage (will be described in Section III-B) to construct and update the set E^t . In this study, E^t is proposed to include both generated data E_{GAN} (based on GAN algorithm [15]) and selected data $E_{herding}$ (based on herding selection method [9]).

$$E^t = [E_{herding}(T^{t-1})]_{\gamma \cdot N_E^t} \cup [E_{GAN}(T^{t-1})]_{N_E^t - \gamma \cdot N_E^t} \quad (1)$$

where γ is the balancing hyperparameter in the exploration-exploitation trade-off. The sets $E_{herding}$ and E_{GAN} are obtained using the herding and GAN techniques to construct the exemplar set.

The target flows set T^t encapsulates the (un)known flows within the target domain S^t , where N_T^t represents the number of flows in the set T^t . In real-world network scenarios, this set inherently comprises applications originating from both source domains S^A (known classes) and new applications (unknown classes). The formulation for T^t is presented as follows:

$$T^t = \left\{ T_k^t \cup T_u^t \mid T_k^t \subset [T_k]_{N_T^t - \alpha \cdot N_T^t} \text{ and } T_u^t \subset \left[\begin{pmatrix} T_u \\ B^t \end{pmatrix} \right]_{\alpha \cdot N_T^t} \right\} \quad (2)$$

where T_k^t , T_u^t denote the sets of known and unknown applications, and B^t unknown applications are added to the S^t target domain. Moreover, to construct a realistic experimental scenario, we distribute applications across target domains encompassing both known and unknown applications, based on the *ratio of unknown class* $\alpha \in [0, 1]$. In this context, when α equals 1, it signifies the inclusion of all known applications from the source domains S^A ; conversely, when α equals 0, it implies that the entire target dataset consists of unknown applications (new applications).

The classification of both T_u^t unknown classes and T_k^t known classes is pursued through the optimization of the Θ^t model on T^t and E^t . Herein Θ^t is distilled from Θ^{t-1} . The number of output neurons in the classifier is expanded to identify B^t new classes. In the target domain (S^t), a flow-feature matrix $x_i^t \in D^t$ is passed through Θ^t to extract automatically distinct features D_k^t via the *Encrypted Traffic Extractor (ETE)* blocks (Section III-C1).

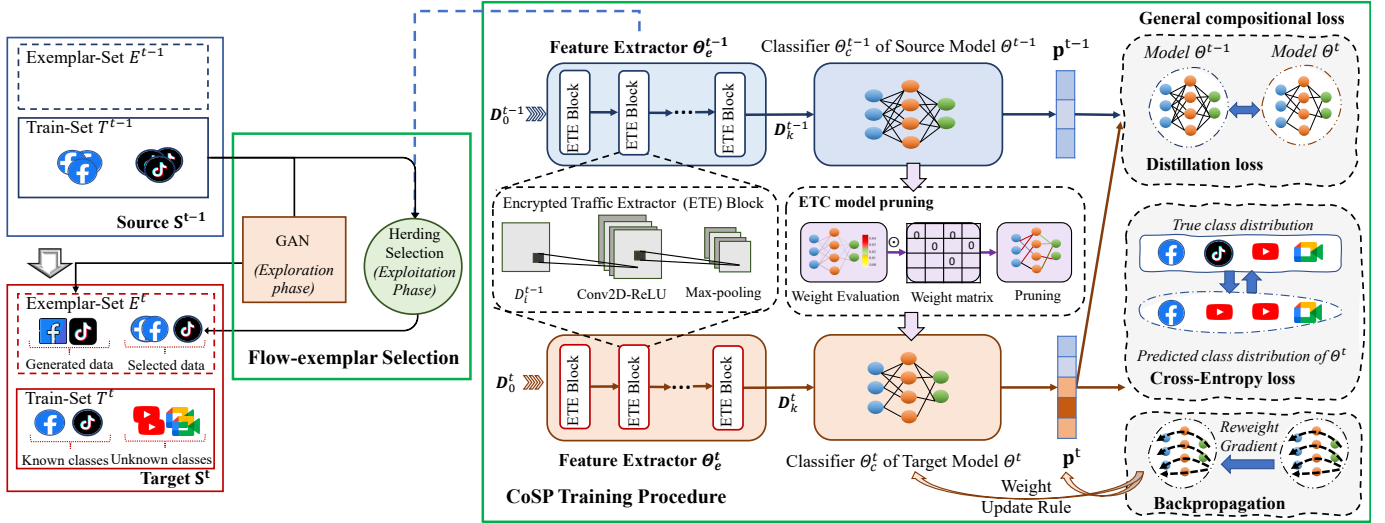


Fig. 2: Illustration overview of our CoSP framework: *Flow-exemplar Selection* block for constructing an exemplar set using two techniques (i.e., Herding selection strategy and GAN flow Augmentation); *Feature Extractor* block Θ_e (including ETE blocks) for automated extraction of important features; *ETC model pruning* block for transferring pruned knowledge from the source model Θ^{t-1} to the target model Θ^t ; and *General compositional loss* (i.e., a distillation loss and a cross-entropy loss) to address the stability-plasticity dilemma.

Subsequently, these features are fed into deep neural network layers for classification. Here, Θ^t represents the model pruned from Θ^{t-1} by weight pruning mechanism to leverage pre-existing domain pruned knowledge and ensure storage capacity as the parameter count increases sharply through *ETC model pruning* block (Section III-C2).

After obtaining probabilities $\mathbf{P}^{t-1}(x_i^t, \Theta^{t-1}) \in \mathbb{R}^{D^{t-1}}$ and $\mathbf{P}^t(x_i^t, \Theta^t) \in \mathbb{R}^{D^t}$, the *general compositional loss* (Section III-C3) is employed through two plug-and-play losses. The first one is the distillation loss $\mathcal{L}_{KD}$ transfers knowledge from Θ^{t-1} (source model) to Θ^t (target model) while ensuring stability in representations of known classes during unknown learning. The second loss is cross-entropy loss $\mathcal{L}_{CE}$ ensures accurate predictions for both existing and newly introduced classes, thereby upholding the model's classification performance across the entire dataset.

Overall, in the SDN domain S^0 (a.k.a first task), encrypted traffic data D_0^0 is learned by optimizing Θ^0 solely through $\mathcal{L}_{CE}$. Continuously learning, in the SDN domain S^t ($t \geq 2$), the objective equation $\bar{\mathcal{L}}_{obj}$ to optimize Θ^t is formulated on Equation 4. After computing $\bar{\mathcal{L}}_{obj}$, its gradients are propagated backward through the network to update the weights using the backpropagation algorithm.

B. Flow-exemplar Selection

One effective strategy for addressing the catastrophic forgetting issue is to reuse a portion of the old applications during incremental training (within the context of SDN domains allowed to access cross-domain data). This study considers the exemplar set E^t as a form of term memory. The ratio of E^t to D^t represents the trade-off between the costs of retraining from scratch and the forgetting of known classes. The construction and update of E^t are governed by an explore-

exploit trade-off parameter γ . Algorithm 1 presents a detailed step-by-step procedure for constructing the E_t in the target domain S^t :

- Line 4: Determining the number of flows in the set E^t .
- Lines 5-10: Herding selection strategies (a.k.a. exploitation) are employed to select representative flows from the source domains.
- Lines 11-17: GAN flow augmentation (a.k.a. exploration) is utilized to explore new features for the training set.

The herding selection strategy leverages the feature extractor Θ_e^{t-1} trained from the source domain S^{t-1} to compute the average feature vector of all exemplars (Line 6), followed by selecting $\gamma \cdot N_E^t$ samples based on the nearest-mean-of-exemplars classification strategy (Line 9).

GAN traffic flow augmentation generates synthetic flows from existing data using a Generative Adversarial Network (GAN). This process involves a generator and discriminator network working in tandem. The Generator Network ($\mathbb{G}$) is constructed by updating the weights ϕ_G to produce flows indistinguishable from real flows (Line 16). Meanwhile, the Discriminator Network ($\mathbb{D}$) learns to classify between real flows and generated flows produced by $\mathbb{G}$ (Line 15). Finally, set E^t is obtained from $\gamma \cdot N_E^t$ selected flows and $N_E^t - \gamma \cdot N_E^t$ generated flows (Line 18).

C. CoSP Training Procedure

1) Encrypted Traffic Extractor

After constructing the input D_0^t in target SDN domain S^t , crucial features D_k^t are automatically extracted through the Encrypted Traffic Extractor (ETE) blocks Θ_e^t . Building upon our previous research [16], which evaluated the performance of various DL-based ETC models using payload-based features, we adopt a CNN architecture featuring five convolutional

Algorithm 1: CoSP Flow-Exemplar Selection

- 1 **Input:** Source training flows T^{t-1} (N_T^{t-1} known classes); N_T^t (Number of target training samples); β (ratio E^t within D^t); γ (ratio explore-exploit data); and N_G (Number of epochs in GAN);
- 2 **Require:** Θ_e^{t-1} (Feature Extractor); $\mathbb{G}$ (Generator model); ϕ_G (Generator parameters); $\mathbb{D}$ (Discriminator model), ϕ_D (Discriminator parameters); // **current model parameters**
- 3 **Output:** Exemplar-set E^t ;
- 4 Set $N_E^t = \beta \cdot N_T^t$ // The ratio E^t within D^t ;
- 5 // **Herd selection Strategies (a.k.a. exploitation)**
- 6 $\mu \leftarrow |N_T^{t-1}|^{-1} \sum_{x \in T^{t-1}} \Theta_e^{t-1}(x)$;
- 7 $E_{herding}^t \leftarrow \{e_i\}_{i=1}^{\gamma \cdot N_E^t}$; // i.e. keep first $\gamma \cdot N_E^t$ samples
- 8 **for** $k \leftarrow 1$ **to** $\gamma \cdot N_E^t$ **do**
- 9 $e_k \leftarrow \underset{x \in T^{t-1}}{\operatorname{argmin}} \left\| \mu - |k|^{-1} \left[\Theta_e^{t-1}(x) + \sum_{i=1}^{k-1} \Theta_e^{t-1}(e_i) \right] \right\|$;
- 10 $E_{herding}^t \leftarrow \{e_i\}_{i=1}^{\gamma \cdot N_E^t}$;
- 11 // **GAN Flow Augmentation (a.k.a. exploration)**
- 12 **for** $k \leftarrow 1$ **to** N_G **do**
- 13 Noise samples $Z_G = \{z_i\}_{i=1}^{N_E^t - \gamma \cdot N_E^t} \sim p_g(Z_G)$;
- 14 Real samples $X_G = \{x_i\}_{i=1}^{N_E^t - \gamma \cdot N_E^t} \sim p_{data}(X_G)$;
- 15 $\phi_D = \phi_D - \nabla \phi_D \cdot \frac{1}{N_E^t - \gamma \cdot N_E^t} \sum_{i=1}^{N_E^t - \gamma \cdot N_E^t} [\log(\mathbb{D}(X_G)) + \log(1 - \mathbb{D}(\mathbb{G}(Z_G)))]$;
- 16 $\phi_G = \phi_G - \nabla \phi_G \cdot \frac{1}{N_E^t - \gamma \cdot N_E^t} \sum_{i=1}^{N_E^t - \gamma \cdot N_E^t} \log(1 - \mathbb{D}(\mathbb{G}(Z_G)))$;
- 17 $E_{GAN}^t \leftarrow \mathbb{G}(\{Z_G\}_{i=1}^{N_E^t - \gamma \cdot N_E^t})$
- 18 **Return** $E^t = E_{herding}^t \cup E_{GAN}^t$

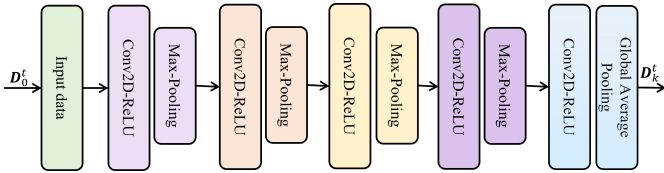


Fig. 3: Illustration the Encrypted Traffic Extractor (ETE) blocks (comprising pairs of Conv2D-ReLU and Pooling) within the *Feature Extractor* phase for the Encrypted Traffic Classification problem using payload-based features.

layers (32, 64, 128, 256, and 512 filters), 3D kernels, and four max-pooling operations (as illustrated in Fig. 3) as our baseline model due to its superior performance.

2) ETC model pruning: Knowledge distillation from pruned source model to target model

To conserve training efforts within the targeted SDN domain, a pruning strategy (e.g., weight pruning [17]) is employed to optimize the weights W^t within the target model Θ_e^t . A binary pruning mask $\mathcal{F}$ is utilized to prune weights as $\tilde{W}^{(t)} = W^{(t-1)} \odot \mathcal{F}^{(t-1)}$. A threshold ξ is used as a weight

pruning threshold, whereby $\mathcal{F}$ is computed as follows:

$$\mathcal{F}_{ij} = \begin{cases} 0, & \text{if } |W_{ij}| < \xi^{(l)} \\ 1, & \text{otherwise} \end{cases} \quad (3)$$

where $\tilde{W}$ denotes the pruned connection weight, $\odot$ denotes the Hadamard product operator¹, and W represents the pre-trained weights of model Θ_c^{t-1} .

3) General Compositional Loss

The loss of the target model Θ^t is evaluated upon the (composite) loss function $\overline{\mathcal{L}}_{obj}$:

$$\overline{\mathcal{L}}_{obj} = \frac{1}{2} \left((1 - \lambda_1) \mathcal{L}_{KD} + \lambda_1 \mathcal{L}_{CE} + \lambda_2 \sum_{i=0}^{N_t} w_i^2 \right) \quad (4)$$

where $0 \leq \lambda_1, \lambda_2 \leq 1$ are balancing hyperparameter. The λ_1 is utilized as a convex combination to balance between the distillation and classification terms. The λ_2 serves as an L2 regularization² parameter to govern the intensity of regularization and individual weight w_i .

The distillation term $\mathcal{L}_{KD}$ aims to preserve previously acquired knowledge, thus preventing weight drift during incremental training (enhancing its stability). $\mathcal{L}_{KD}$ is formulated as:

$$\begin{aligned} \mathcal{L}_{KD} &= |N_t|^{-1} \sum_{i=1}^{N_t} \mathcal{D}_{KL}(\mathcal{S}(\Theta^{t-1}(x^{t_i})) \parallel \mathcal{S}(\Theta^t(x^{t_i}))) \\ \mathcal{S}(\Theta^{t-1}(x^{t_i})) &= \frac{e^{\hat{p}_i(\Theta^{t-1}(x^{t_i}))/t}}{\sum_{j=1}^{N_t} e^{\hat{p}_j(\Theta^{t-1}(x^{t_i}))/t}}, \\ \mathcal{S}(\Theta^t(x^{t_i})) &= \frac{e^{p_i(\Theta^t(x^{t_i}))/t}}{\sum_{j=1}^{N_t} e^{p_j(\Theta^t(x^{t_i}))/t}} \end{aligned}$$

where $\mathcal{S}(\cdot)$ is the softmax operation, $\mathcal{D}_{KL}(\cdot \parallel \cdot)$ for the Kullback-Leibler divergence (KLD). p_i and $\hat{p}_i$ represent the prediction logits of the i -th application, with t as a temperature scalar set to values greater than 1, thereby assigning larger weights to smaller values.

The cross-entropy loss $\mathcal{L}_{CE}$ (classification term) serves to integrate new incremental knowledge into the model (enhancing its plasticity). $\mathcal{L}_{CE}$ is defined as follows:

$$\mathcal{L}_{CE} = |N_t|^{-1} \sum_{i=1}^{N_t} \mathcal{D}_{CE}(y_i^t, \mathcal{S}(\Theta^t(x_i^t))) \quad (5)$$

where y_i^t is the ground-truth label, $\mathcal{D}_{CE}(\cdot, \cdot)$ indicates cross-entropy loss.

In this study, Stochastic Gradient Descent (SGD) is utilized to optimize the loss function $\overline{\mathcal{L}}_{obj}$. SGD incorporates the concept of momentum, which aggregates gradients from previous steps to determine the direction of optimization. Specifically, SGD with momentum tracks the update $(\Delta\Theta)_{(\cdot)}$ at each iteration, and calculates the subsequent update as a linear combination of the gradient $\nabla \mathcal{L}_{obj}(\Theta_k)$ and the previous update

¹The Hadamard product is the element-wise product of two matrices.

²L2 regularization is a technique in machine learning that prevents overfitting by adding a penalty term to the loss function.

$(\Delta\Theta)_k$ expressed as $\Theta_{k+1} = \Theta_k - \eta \nabla \overline{\mathcal{L}_{obj}}(\Theta_k) + \tau(\Delta\Theta)_k$, where η denotes the learning rate and τ signifies the exponential decay factor. Setting $\tau = 0$ results in standard SGD with no momentum. Recommended momentum values of $\tau = 0.9$ are advocated in prior works [10].

IV. EXPERIMENTS AND RESULTS

This section presents a comparative evaluation of our CoSP framework against state-of-the-art (SoA) IL methods across various benchmark scenarios. Additionally, ablation studies are conducted to evaluate the individual contributions of CoSP components and their sensitivity to hyperparameter variations.

A. Experimental Settings

1) Dataset

This study utilizes the Netflow-GQUIC dataset [18] and a self-collected dataset to evaluate the performance of the proposed CoSP-based encrypted traffic classification method. The ‘Netflow-GQUIC dataset’³ captured over a three-week period in March 2018. Our research emphasis lies in facilitating incremental learning of encrypted traffic, necessitating the diversification of encrypted traffic classes to verify the robustness of CoSP. The ‘self-collected dataset’⁴ incorporates traffic data from commercial websites in Southeast Asia and other platforms.

2) Input definition

The raw traffic flow is preprocessed into a flow-level feature matrix X with dimensions $N_p \times N_B$, defined as $X = [\{b_i^j\}_{j=1}^{N_B}]_{i=1}^{N_p}]^T$. The notation $[\cdot]^T$ denotes the transpose function, b_i represents the packet byte values, and N_B and N_p , respectively, represent the number of bytes per packet and the number of packets per flow. In the work [19], it was demonstrated that a network traffic flow with the first 20 packets ($N_p = 20$) and 245 bytes encrypted in each packet ($N_B = 256$) achieved the best classification performance.

3) Evaluation metrics

This part outlines the accuracy-based metrics used in our evaluation. The accuracy, $Acc(\Theta_t)$, is computed for each training session on a specific task (episode), where both known and unknown classes are classified. The average accuracy, $Avg = |n|^{-1} \sum_{t=0}^n Acc(\Theta_t)$, is calculated as the mean accuracy across all experimental episodes. Last Accuracy, $Last = Acc(\Theta_n)$, is the accuracy at the last episode (a.k.a target SDN domain). The forgetting metric evaluates the level of catastrophic forgetting based on the average difference between $Acc(\Theta_0)$ and the accuracy on each episode, as defined:

$$\text{Forgetting} = |n - 1|^{-1} \sum_{t=1}^n \mathbb{I}[Acc(\Theta_0) > Acc(\Theta_t)] \cdot (Acc(\Theta_0) - Acc(\Theta_t)) \quad (6)$$

where $\mathbb{I}[\cdot]$ is the Iverson bracket, which equals 1 if the condition inside is true and 0 otherwise.

³**Netflow-GQUIC dataset:** Youtube (4645 flows), Google Hangout Chat (2556 flows), and Google Hangout VoIP (2198 flows).

⁴**Self-collected dataset:** Thegiodidong (509 flows), Shopee (2581 flows), Tiki (1205 flows), Amazon (834 flows), Alibaba (1401 flows), Tiktok (681 flows), Facebook (1379 flows), eBay (1836 flows), and File Transfer (1569 flows).

4) CoSP configuration

The parameters of our CoSP method presented in the previous section can be represented as a tuple of $\text{CoSP}(\beta, \gamma, \eta, \xi, \lambda_1, \lambda_2)$, where β represents the ratio between the number of samples in the target domain Θ^t and the source domains Θ^{t-1} , γ signifies the ratio between the number of generated samples and selected samples, η denotes the learning rate, ξ stands for the pruning threshold, λ_1, λ_2 serve as hyperparameters to balance the composite loss function. We leverage the optimized hyperparameters from [19], [17], [9] with $\eta = 0.001$, $\xi = 0.25$, $\lambda_1 = 0.55$, and $\lambda_2 = 0.45$. The impact of varying β and γ values will be evaluated in Section IV.

5) IL scenarios

A syntax $\text{Base}\langle A \rangle + \text{Step}\langle B \rangle$ is referenced as a task involving the incremental addition of B applications to a model that already includes A applications. Considering the 12 applications within the two datasets (Section IV-A1), two distinct IL scenarios are investigated, outlined as follows:

- **Training from Scratch (TFS):** The $\text{Base0} + \text{Step3}$ scenario involves the network system starting from scratch. In each episode thereafter, an additional 3 unknown classes are introduced.
- **Training from Half (TFH):** To complement the previous setup, an additional scenario, $\text{Base6} + \text{Step}\{1, 2, 3\}$, is introduced, where pre-training is conducted on a specific set of labels (corresponding to real-world ETC scenarios). This scenario requires the IL models to adapt flexibly to varying numbers of added unknown classes until reaching a total of 12 applications.

B. Results

This part evaluates the performance of CoSP through comprehensive comparisons with SoA benchmarks across the two IL scenarios. To further understand CoSP’s behavior, we investigate the impact of data imbalance and exploitation. Finally, we assess CoSP’s adaptability to varying numbers of unknown classes and compare its performance to SoA methods.

1) Overall comparison

Table I illustrates the comprehensive comparative analysis of our CoSP model and other IL techniques applied to the GQUIC dataset and a self-collected dataset, focusing on metrics including Avg, Last, and Forgetting performance defined in Section IV-A3. At first glance, the performance of most algorithms under the TFH scenario (most relevant to ETC) is superior to those trained from scratch under the TFS scenario. Our CoSP method consistently exhibits superior performance compared to benchmarks across both scenarios. Specifically, CoSP demonstrates improvements of -0.07% and +0.24% in terms of average accuracy (higher values are better), +0.68% and +1.07% in terms of last accuracy (higher values are better), and -0.54% and -0.94% in terms of Forgetting (lower values are better) compared to the best-performing benchmark approaches (FOSTER) in TFS and TFH scenarios, respectively.

TABLE I: Comprehensive evaluation of **Average accuracy (Avg)**, **Last accuracy (Last)**, and **Forgetting** of CoSP compared to SoA IL methods across two scenarios (i.e., Training from scratch (TFS) and Training from half (TFH)). The last row illustrates the improvement of the proposed CoSP vs. the best-performing benchmark approaches, highlighting superior (**bold green**) and inferior (**bold gray**) performance. The up arrow ($\uparrow$) indicates higher values are preferable, whereas arrows ($\downarrow$) indicate lower values are preferable. Results are reported as averages over 30 runs.

Approaches	TFS: Base0+Step{3}			TFH: Base6Step{1,2,3}		
	Avg $\uparrow$	Last $\uparrow$	Forgetting $\downarrow$	Avg $\uparrow$	Last $\uparrow$	Forgetting $\downarrow$
Fine-tune	70.76	49.25	35.80	73.27	56.61	17.72
Replay	72.75	51.28	32.74	73.74	57.81	16.88
LwF	75.15	52.53	29.81	76.51	62.21	14.79
iCaRL	83.21	71.72	18.18	77.97	66.18	11.39
DER	87.37	79.21	13.42	81.42	75.61	6.72
POSTER	88.08	80.93	12.28	82.26	76.61	6.33
CoSP (ours)	88.01	81.61	11.73	82.50	77.68	5.39
Improvement	(-0.07)	(+0.68)	(-0.54)	(+0.24)	(+1.07)	(-0.94)

Fig. 4 presents the average accuracy variations of different methods following each incremental episode on TFS (Fig. 4a) and TFH (Fig. 4b). At the first episode, the accuracies of the approaches are comparable, standing at 96% and 87% in TFS and TFH, respectively. However, the addition of new SDN domains (resulting in the emergence of unknown classes) results in a notable decline in algorithmic accuracies (Data-Centric and Algorithm-Centric categories). Remarkably, Fine-tuning experiences a drastic drop, reaching below 50% accuracy at the last episode, representing a twofold reduction compared to the first episode. Conversely, Model-Centric algorithms demonstrate a more gradual decline in accuracy. Our CoSP methods exhibit minimal accuracy variance, maintaining 96.81% (first episode) to 81.61% (last episode) in TFS, and 86.55% (first episode) to 77.68% (last episode) in TFH, respectively.

2) Impact of Data Imbalance and Exploitation

Two hyperparameters exist in CoSP: the Old-New Imbalance Ratio β , and the Explore-Exploit Tradeoff Coefficient γ . Experimental scenarios (i.e., TFH scenario) are evaluated with varying adjustments of the Old-New Imbalance Ratio β from $\{5, 10, 25, 55, 80, 100\}\%$, and the Explore-Exploit Tradeoff Coefficient γ from $\{10, 20, 50, 70, 100\}\%$. Subsequently, the average accuracies is evaluated based on the impact of varying these parameters on CoSP's performance, as illustrated in Fig. 5. Specifically, when $\beta = 5\%$ and $\gamma = 10\%$ —representing an exemplar size of 5% relative to the target domain samples and utilizing only 10% of samples chosen from the herding strategy—the accuracy of CoSP is minimally achieved, at approximately 64.38%. Conversely, the highest accuracy, reaching 83.51%, is observed when β is 55% and γ is 70%. Hence, we suggest to use these values of β and γ parameters for this version of the CoSP method.

3) Impact of the number of unknown classes

As illustrated in Fig. 6, the ratio of unknown classes is gradually increased with α ranging from 5% to 50%, and the performance of the IL methods on the TFH scenario is recorded. At $\alpha = 5\%$, insignificant variance is observed in the accuracy of most algorithms, with CoSP achieving the highest accuracy of 82.51%. As the number of unknown

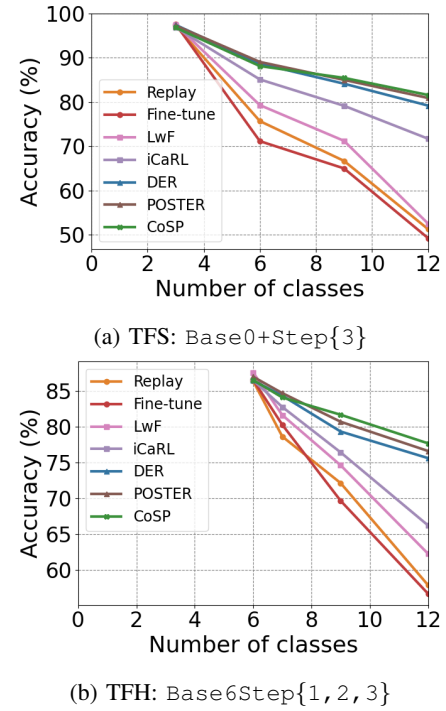


Fig. 4: Performance on average accuracy of CoSP compared to SoA IL methods across each incremental episode of: (a) Training from scratch (TFS, starting from 3 base classes); (b) Training from half (TFH, starting from 6 base classes).

classes increases (from 5% to 50%), a sharp decline in accuracy is experienced by Fine-tune and DER, decreasing from 73.27% to 60.02% and 81.45% to 76.14%, respectively, while CoSP demonstrates a notable increase to 87.11%. From these results, it is evident that with the effective construction of the exemplar-set, the proposed CoSP model can effectively adapt to a large number of unknown classes.

V. CONCLUSIONS

This study introduces CoSP, a novel IL framework specifically designed to address the challenges of evolving ETC models in dynamic SDN environments. By incorporating a

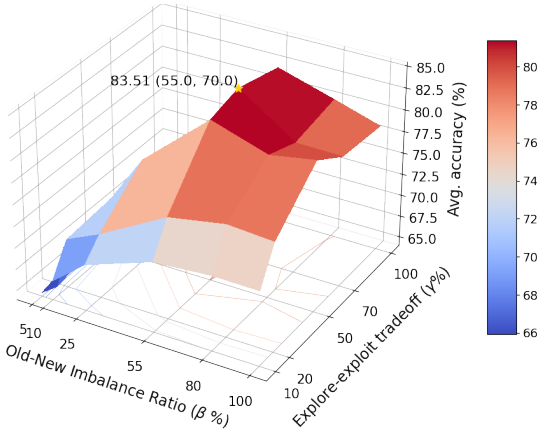


Fig. 5: Evaluating the Average accuracy of CoSP under various Old-New Imbalance Ratios $\beta(\%) \in \{5, 10, 25, 55, 80, 100\}$, and Explore-Exploit Tradeoff Coefficients $\gamma(\%) \in \{10, 20, 50, 70, 100\}$ across the Training from Half (TFH) scenario. The gold star (*) denotes the highest accuracy value.

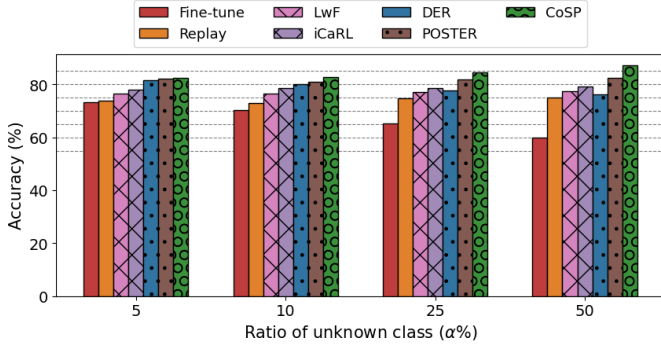


Fig. 6: Evaluating the Average accuracy of the proposed CoSP vs. SoA IL methods across varying numbers of **unknown classes** α (ranging from 5% to 50%) across the Training from Half (TFH) scenario.

flow-exemplar selection algorithm, model pruning, and a dual-loss function, CoSP effectively balances accuracy and forgetting to mitigate catastrophic forgetting. Extensive evaluations across diverse datasets and IL scenarios demonstrate CoSP's superiority over existing IL frameworks, achieving significant improvements in both forgetting and handling increased unknown classes. Our findings highlight the potential of CoSP to enhance the adaptability and resilience of ETC systems in the face of evolving network traffic patterns. By providing a robust and adaptable solution to the ETC problem, CoSP contributes to the advancement of network security and intelligence. Future research will focus on addressing heterogeneous forgetting through gradient-based approaches to further optimize the framework's performance.

VI. ACKNOWLEDGEMENT

Son Duong's study was funded by the Master and PhD Scholarship Program of Vingroup Innovation Foundation

(VINIF), code VINIF.2023.ThS.113.

REFERENCES

- [1] G. W. Geremew and J. Ding, "Elephant flows detection using deep neural network, convolutional neural network, long short term memory and autoencoder," *arXiv preprint arXiv:2306.03995*, 2023.
- [2] "Let's Encrypt," <https://letsencrypt.org/>, accessed: April 3, 2024.
- [3] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [4] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.
- [5] H. Y. He, Z. G. Yang, and X. N. Chen, "Pert: Payload encoding representation from transformer for encrypted traffic classification," in *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, 2020, pp. 1–8.
- [6] Y. Maleh, Y. Qasmaoui, K. El Gholami, Y. Sadqi, and S. Mounir, "A comprehensive survey on sdn security: threats, mitigations, and future directions," *Journal of Reliable Intelligent Environments*, vol. 9, no. 2, pp. 201–239, 2023.
- [7] K. Zhu, Y. Cao, W. Zhai, J. Cheng, and Z.-J. Zha, "Self-promoted prototype refinement for few-shot class-incremental learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 6801–6810.
- [8] X. Li, J. Xie, Q. Song, Y. Sang, Y. Zhang, S. Li, and T. Zang, "Let model keep evolving: Incremental learning for encrypted traffic classification," *Computers & Security*, vol. 137, p. 103624, 2024.
- [9] G. Bovenzi, A. Nascita, L. Yang, A. Finamore, G. Aceto, D. Ciuonzo, A. Pescapé, and D. Rossi, "Benchmarking class incremental learning in deep learning traffic classification," *IEEE Transactions on Network and Service Management*, 2023.
- [10] Z. Wu, Y.-n. Dong, X. Qiu, and J. Jin, "Online multimedia traffic classification from the qos perspective using deep learning," *Computer Networks*, vol. 204, p. 108716, 2022.
- [11] Y. Chen, T. Zang, Y. Zhang, Y. Zhou, L. Ouyang, and P. Yang, "Incremental learning for mobile encrypted traffic classification," in *ICC 2021-IEEE International Conference on Communications*. IEEE, 2021, pp. 1–6.
- [12] I. Lee, H. Roh, and W. Lee, "Encrypted malware traffic detection using incremental learning," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2020, pp. 1348–1349.
- [13] T. Shapira and Y. Shavitt, "Flowpic: A generic representation for encrypted traffic classification and applications identification," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1218–1232, 2021.
- [14] H.-K. Lim, J.-B. Kim, K. Kim, Y.-G. Hong, and Y.-H. Han, "Payload-based traffic classification using multi-layer lstm in software defined networks," *Applied Sciences*, vol. 9, no. 12, p. 2550, 2019.
- [15] Cao, "Filter-gan: Imbalanced malicious traffic classification based on generative adversarial networks with filter," *Mathematics*, vol. 10, no. 19, 2022. [Online]. Available: <https://www.mdpi.com/2227-7390/10/19/3482>
- [16] N.-T. Hoang, C.-S. Duong, M.-N. Vu, H.-H. Nguyen, X.-T. Tran, V. Tong, and H. A. Tran, "A new transfer learning-based traffic classification algorithm for a multi-domain sdn network," in *Proceedings of the 12th International Symposium on Information and Communication Technology*, 2023, pp. 235–242.
- [17] C. Mao, Q. Liang, C. Pan, and I. Schizas, "A statistical approach for neural network pruning with application to internet of things," *EURASIP Journal on Wireless Communications and Networking*, vol. 2023, no. 1, pp. 1–21, 2023.
- [18] V. Tong, H. A. Tran, S. Souihi, and A. Mellouk, "A novel quic traffic classifier based on convolutional neural networks," in *2018 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2018, pp. 1–6.
- [19] C. Dao, V. Tong, N.-T. Hoang, H.-A. Tran, and T. X. Tran, "Enhancing encrypted traffic classification with deep adaptation networks," in *2023 IEEE 48th Conference on Local Computer Networks (LCN)*. IEEE, 2023, pp. 1–4.